AFRINEXT · PHASE 5 · OBJECT STORAGE

Two Machines, One Bucket, Nothing Else Changed

A file uploaded by one web instance is not on another instance's disk. Until this milestone, Afrinext could only ever run on one server without a buyer's download failing depending on which machine answered. The fix is an object-storage adapter behind the port that already existed — and the point of the work is everything that **did not** move: not one line of the authorization chain.

Branch **claude/create-app-repository-1cxsih** | HEAD **2eeb148** | Unit tests **668 passed** | Browser tests **46 passed** | Mutations killed **23 / 23** | Migrations **none** | Ledger touched **none**

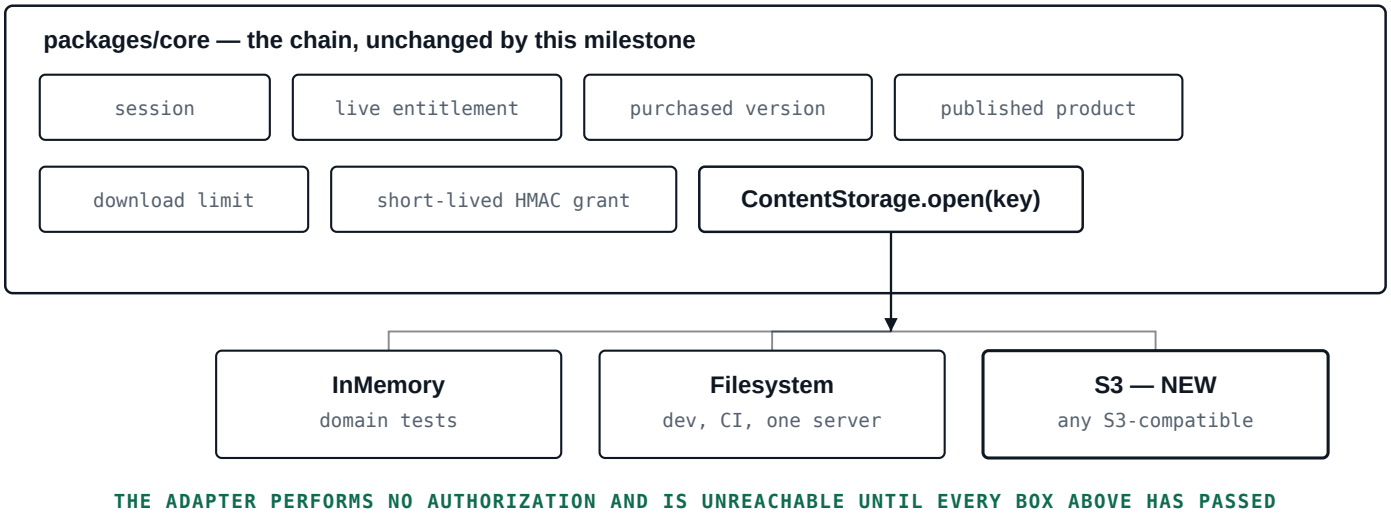
THE MILESTONE IN ONE PARAGRAPH

`ContentStorage` gained a third implementation, `S3ContentStorage`, which speaks the S3 HTTP API to any S3-compatible provider. The in-memory and filesystem adapters are untouched and still used. **No durable public URL exists anywhere in the design** — `open()` returns a `Buffer`, authenticated server-to-server, so there is no pre-signed link to leak, log or share. The authorization chain — session → live entitlement → purchased version → published product and store → download limit → short-lived HMAC grant → `ContentStorage.open()` — is byte-identical to what Phase 5 shipped, and a test proves both adapters produce the same answers to all fourteen of its observable outcomes.

No credentials, no bucket, no bill. `CONTENT_STORAGE` is not set to `s3` anywhere, including the Render preview. The adapter is real code tested against a real HTTP server that verifies its signatures; what does not exist is an account, and nothing here pretends otherwise.

A The boundary was already there. It was not moved.

Phase 5 defined a port with two properties chosen for exactly this moment: **keys are opaque** (a key is not a URL and cannot be turned into one by a client) and **reads return bytes, not locations**. Adding object storage was therefore a new class, not a change to any caller.



Three implementations of one port. The chain above them is the same code it was before this milestone.

The adapter is **S3-compatible, not AWS-only**. One implementation serves AWS S3, Cloudflare R2, Backblaze B2, Scaleway, DigitalOcean Spaces and MinIO, because they implement the same API. A provider is chosen by setting an endpoint, not by shipping different code — which matters most at the moment vendor lock-in usually bites, when egress pricing changes.

B There is no pre-signed URL, and that is deliberate

The brief permitted pre-signed URLs provided they were minted only after authorization and were short-lived. **This implementation does not mint one at all**, which is strictly stronger, and the reasoning is worth stating because the weaker option is the industry default.

> **WHAT A PRE-SIGNED URL WOULD COST**
>
> A pre-signed URL is a bearer credential for the object. Once created it is valid for whoever holds it: it survives being pasted into a chat, appearing in a browser history, being captured by a CDN or proxy log, or being forwarded. Its lifetime cannot be

shortened after the fact, and the object store honours it without consulting Afrinext — so a revoked entitlement, a refunded order or a suspended store **cannot stop it**. Every one of those is a rule this engine enforces.

Instead, `open()` performs an authenticated `GET` from the server and reads the response into a `Buffer`. The browser keeps receiving bytes from Afrinext's own origin, through the one route that demands a session and an unexpired grant. The cost is bandwidth through the application; the benefit is that **access remains revocable at every instant**, which is the property the refund and suspension rules depend on.

PROPERTY	PRE-SIGNED URL	WHAT THIS SHIPS
Valid for	anyone holding the string	the session that was authorized
After a refund revokes entitlement	still works until expiry	refused immediately
After the store is suspended	still works until expiry	refused immediately
Leaks through logs, history, chat	yes, it is the whole URL	nothing to leak
Counts against the download limit	once, at mint time	once, at delivery
Bucket name reaches the browser	yes	no

A LEAK THIS MILESTONE INTRODUCED, AND CLOSED

The first version of the adapter put the provider's response body into the error it threw on a failed upload, with a comment claiming that error was only ever logged. It is not: the server action that attaches a file renders `error.message` straight onto the seller's screen, so any seller who could make an upload fail would have been shown the bucket name, the object key and an account id.

The detail is now logged under `component: content.s3` and the thrown `StorageWriteFailedError` says only that the file could not be saved. A test asserts the bucket, the key, the endpoint host and the provider's status are all absent from what the seller reads, and mutation **S5** restores the old shape to prove the test catches it. **Reads were never affected** — the download route collapses every refusal into one generic 403.

The last row is asserted by a browser test: after a real purchase, the library page and the product page are fetched and their HTML checked to contain neither the `storage_key`, nor the bucket name, nor the endpoint.

C Signature Version 4, written out and checked against AWS's own answer

`packages/core` has five runtime dependencies on purpose. This adapter needs three verbs on one object, and SigV4 is a published, closed specification: a canonical request, a string to sign, a derived key, an HMAC. Sixty lines of `node:crypto` is the same posture the codebase already takes with its own HMAC content grants and its hand-written SQL.

The risk is real — a subtly wrong signature fails only against a live server — so it is answered twice:

1. **AWS's published test vector.**

   `signV4` is asserted to reproduce, exactly, the `Authorization` header from Amazon's own worked example, including the `Range` header and the signed-header list. That checks canonicalisation and key derivation against a known-good answer rather than against one reading of the specification.

2. **A server that verifies.**

   Every adapter test runs against an HTTP server that independently recomputes the signature from the request it received and answers `403` on a mismatch. It also verifies `x-amz-content-sha256` against the body, so a request whose payload hash disagrees with its payload is refused rather than accepted.

A third test changes one signed element at a time — the object path, the verb, the region, the date, the secret — and asserts the signature changes each time. A signature that ignores an input is a signature that can be replayed against a different object.

D "Nothing else changed" is a claim, so it is a test

An adapter that returned a slightly different content type, or failed in a way the download counter read as success, would leave every existing test green while changing what a buyer receives. So the same journey runs **twice**, once per adapter, through the same domain functions, and the two runs are compared field by field.

The journey: a seller opens a store, publishes it, creates a product, attaches a file, sets a download limit of two, writes licence text and publishes. A buyer checks out, pays through the mock provider, and downloads. The seller then publishes a *second* version.

The buyer downloads again, and again. A stranger tries. The order is refunded. Fourteen observations are recorded:

OBSERVED	VALUE, ON BOTH ADAPTERS
bytes served	`version one`
content type	`application/pdf`
asset title, disposition	as attached
version owned	`1`
licence snapshot	`Usage personnel.`
download limit	`2`
a stranger downloading	`ContentForbiddenError`
bytes after v2 is published	`version one`
the buyer requesting v2's file	`ContentForbiddenError`
second download	`ok`
third download	`DownloadLimitReachedError`
after the refund	`ContentForbiddenError`
rows in `entitlement_downloads`	`2`

The test asserts the two records are equal *and* that the values are the right ones — two adapters broken the same way would pass equality alone.

> **THE ONE THAT MATTERS MOST: A STORAGE OUTAGE MUST NOT COST A DOWNLOAD**
>
> A second test makes the object store answer `503`. The download is refused, **zero rows** appear in `entitlement_downloads`, and the buyer's next attempt succeeds and returns the right bytes with both downloads still available. A buyer who paid must not lose an allowance because a bucket had a bad minute — and the ordering that makes this true is that the delivery is counted after `open()` returns, never before.

E Two application instances, proved through the real screens

This is the milestone's actual promise, so it is tested the way the promise is made rather than by calling a function twice. The browser suite now starts **four** processes: the existing instance on filesystem storage, a signature-verifying bucket, and two further `next start` instances — A and B — on different ports, configured for object storage and sharing only the database and that bucket.

1. A seller signs in on **instance A**, opens a store through the real four-step wizard, publishes it, adds a product, uploads a PDF through the real file input, and publishes.

2. A buyer signs in on **instance B**, buys the product, completes the buyer profile, chooses mobile money, pays, and the mock webhook confirms.

3. The buyer mints an access grant and fetches the file **from instance B**. The response is 200 and the body is byte-for-byte what the seller uploaded on A.

4. The library page and product page on B are checked for the storage key, the bucket name and the endpoint. None appears.

5. A third signed-in stranger on B mints nothing: the request is refused.

Neither instance is given a content directory. Against the filesystem adapter this test cannot pass unless the two processes happen to share a disk — which is precisely the limitation being removed.

> **A DEBUGGING NOTE WORTH RECORDING**
>
> The first run failed with a 403 that looked like a signing bug and was not. Playwright resolves a relative `page.goto` against `use.baseURL`, so the shared sign-in and store-wizard helpers were silently pulling both actors back to the default instance — everything worked, on the wrong server. The helpers now take an explicit origin. A multi-instance test that quietly collapses into a single-instance test is worse than no test, because it reports success.

F The interesting half is what a misconfiguration does

The failure this milestone exists to prevent does not announce itself at startup. It looks like this: a deployment is set to object storage, a variable is missing, the code quietly uses the local disk, the storefront appears healthy for a week, and then one buyer's download breaks on one machine out of two. So every misconfiguration **fails closed, at startup**:

CONFIGURATION	RESULT
`CONTENT_STORAGE=s3`, any of the five settings missing or empty	throws — never falls back to disk
`CONTENT_STORAGE=filesystem` under `NODE_ENV=production`	throws unless `ALLOW_LOCAL_CONTENT_STORAGE=yes`
`ALLOW_LOCAL_CONTENT_STORAGE` set to `true`, `1`, `YES`, `y`	throws — only the exact string counts
`CONTENT_STORAGE=S3` or `r2` or any typo	throws — a typo must not resolve to the default
nothing set, development	filesystem, as before

> **THE MUTATION MATRIX MOVED THIS CODE, AND THAT IS THE FINDING WORTH READING**
>
> This decision was first written inside the Next.js app, where it read `process.env` directly. Mutation **S2** deleted the guard — making a missing bucket fall through to local disk — and **survived**: no unit test could reach a function that only exists in `apps/web`.
>
> It now lives in `packages/core/src/content/select.ts` as `selectContentStorage(env)`, a pure function of an environment object. Each misconfiguration became a two-line test, three further mutations became possible, and `apps/web/src/lib/content.ts` went back to what it should be: one call and a cache. **A fail-closed rule that nothing tests is a rule that quietly stops happening.**

G Mutations

The Phase 5 matrix of eighteen was re-run in full against this HEAD and extended with four storage mutations. Each runs in a detached git worktree against a separate database, with a guard that refuses to start if it shares `TEST_DATABASE_URL` with the checkout.

#	THE LIE THE MUTATION TELLS	RESULT
S1	A refused read from the object store returns an empty file instead of throwing — so the buyer receives zero bytes *and* spends a download.	`killed · 4 failing`
S2	Object storage selected with no bucket quietly falls back to the local disk.	`killed · 3 failing`
S3	An unrecognised adapter name resolves to the default instead of throwing.	`killed · 1 failing`
S4	A production build is allowed to keep uploaded files on one machine's disk without the deliberate flag.	`killed · 6 failing`
S5	A failed upload puts the provider's error body — which names the bucket, the key and an account id — into the message rendered on the seller's screen.	`killed · 3 failing`
D1–D18	The Phase 5 matrix: public files, another buyer's purchase, unbound grants, unpaid orders, revoked entitlements, ignored limits, resetting counters, mutated purchased versions, licence following the head, unbought library rows, caller-supplied store scope, draft and suspended products, invented placeholder files, version pinning in both directions, and both directions of the refund-revocation rule.	`all 18 killed`

TWO THINGS THE MATRIX CAUGHT ABOUT ITSELF

A mutation that survives is not automatically a hole in the tests. S2's first form — object storage selected with no bucket — survived because an incomplete `s3` configuration fell through to the guard that refuses an unrecognised adapter name, which threw anyway. The rule is defended at two independent points, so a single-point mutation proved only that the other one works. S2 now breaks both, which is the only version of that bug that could actually reach a filesystem adapter.

And the matrix must run the whole suite. Narrowing the command to the two directories being mutated looked like a speed-up and let D4 — "an unpaid order grants access" — survive, because the test that kills it lives in `src/orders`. A matrix run against a subset of the tests measures the subset, not the code. It was restored to the full suite and D4 dies again.

```
23 mutations, 23 killed, 0 survivors. Source verified pristine
afterwards.
```

H Results, boundary, and what is next

CHECK	RESULT
`pnpm lint`	clean
`pnpm typecheck`	clean
`pnpm test` (unit + DB integration)	668 passed
`pnpm build`	compiled
`pnpm test:e2e`	46 passed
Mutations	23 / 23
Migrations	none — this milestone adds no schema
CI	3 / 3 green on 2eeb148

WHAT THIS MILESTONE DID NOT TOUCH

No ledger row, no settlement, no seller payout, no wallet movement, no commission payment, no referral payout, no change to payment capture, no admin console, no seller-dashboard redesign. **iPayMoney remains unimplemented and its sandbox remains pending.** The refund domain is called by the equivalence test, not extended.

OPEN, AND NEEDING A DECISION FROM YOU

1. No provider is chosen and no bucket exists. The adapter is built and tested; an account, credentials and a bill are an operational decision that is not made here. Cloudflare R2 is the obvious candidate for this workload — zero egress fees, and egress is what a download-heavy catalogue pays for — but that is a recommendation, not a decision taken.

2. Bytes flow through the application. That is the price of keeping access revocable, and it is the right trade at Afrinext's size. It becomes a cost worth revisiting at a scale that does not exist yet, and revisiting it would mean accepting the rows in section B's table.

3. The Render preview still runs filesystem storage on its mounted disk, deliberately, with `ALLOW_LOCAL_CONTENT_STORAGE=yes`. It is one instance, so that is correct — and it stops being correct the moment a second instance is added.

Next milestone, if approved: choose an object-storage provider and issue credentials, so the adapter that is already built and tested has a bucket to talk to. Nothing else in this milestone is waiting on anything.